

PIWM 车道(Lane)预测机制详解

Resample-based lane propagation —— 对应代码 `models/dynamics_lane_v6.py`

1. 整体目标

Encoder 从图像里预测一个 29 维的 latent: 9 维车体状态 + 20 维车道。20 维车道由 10 个 waypoint 组成, 每个 waypoint 是 body 系下的二维点 (x_b, y_b) 。第 i 个 waypoint 永远表示“沿车道中心线、距车 s_i 处的点”, 其中 $s_i \in \{-5, 0, 5, \dots, 40\}$ (共 10 个, 间距 $\Delta s=5$)。

Dynamics 块的 lane 分支负责回答一个问题: 给定 t 时刻的 20 维车道、车的物理状态、动作 a_t , 如何输出 $t+1$ 时刻的 20 维车道, 并且保证“第 i 个槽位仍然代表沿路 s_i 处”? v6 的解法分四步, 每步只做一件可解释的事情。

2. 符号表

符号	维度 / 单位	含义
z_t^{car}	11 (归一化)	t 时刻车体物理状态: $[x, y, \psi, v_x, v_y, \omega, w_0, w_1, w_2, w_3, \delta]$, 分别为世界系位置、yaw、世界系速度、yaw 角速度、四个轮速、转向角
z_t^{lane}	20 (归一化)	t 时刻车道 waypoint 拼接, 即 $(x_b^0, y_b^0, \dots, x_b^9, y_b^9)$
a_t	3	t 时刻动作 (steer, gas, brake)
w_i	$(x_b, y_b) \in \mathbb{R}^2$	第 i 个 waypoint (t 时刻 body 系下), $i = 0..9$
s_i	弧长(单位)	第 i 个槽位的目标弧长偏移, 固定为 $\{-5, 0, 5, \dots, 40\}$
Δs	5 (单位)	相邻槽位的弧长间距 (LANE_S_SAMPLES 步长)
Δt	1/50 s	仿真步长 (50 FPS, DT in config.py)
v_x, v_y	世界系	车在世界系下的纵/横向速度分量 (由 z_t^{car} 反归一化得到)
ψ (psi)	rad	车的 yaw 角 ($z_t^{\text{car}}[2]$)
ω (omega)	rad/s	yaw 角速度 ($z_t^{\text{car}}[5]$)
c, s	标量	$\cos(\psi), \sin(\psi)$, 反复出现的简写
Δx_b	标量	一步内车在 body 系下的横向位移分量
Δy_b	标量	一步内车在 body 系下的前进位移分量
$\Delta \psi$	标量	一步内车的 yaw 增量 $= \omega \cdot \Delta t$
$R(\theta)$	2x2 旋转矩阵	标准平面旋转, $R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$
w'_i	$(x_b, y_b) \in \mathbb{R}^2$	Step 1 后, 在 $t+1$ 时刻 body 系下表达的同一个世界点
w_i	$(x_b, y_b) \in \mathbb{R}^2$	Step 2 后, 重采样得到的、对应槽位 i 的新 waypoint

α (alpha)	无量纲	重采样插值系数 $\alpha = \Delta y_b / \Delta s$
MLP_res	23→20	lane 残差网络:输入 $[z_{t+1}^{car}, w$ (归一化), $a_t]$

3. 预备:计算车的 body 系单步位移

在做任何车道几何之前,先把车自己的运动从世界系翻译到 body 系。这部分代码在 dynamics_lane_v6.py 第 64-74 行:

$$\Delta x_b = (v_x \cos \psi + v_y \sin \psi) \Delta t$$

$$\Delta y_b = (-v_x \sin \psi + v_y \cos \psi) \Delta t$$

$$\Delta \psi = \omega \Delta t$$

注意 PIWM 约定:body+y 是前进方向,所以 Δy_b 是车沿车道走的距离; Δx_b 是横向漂移(漂移过弯时不为零); $\Delta \psi$ 是车自身转过的角度。三者一起完整描述“一个 Δt 后,新的 body 系相对于老 body 系的位姿”。

4. Step 1 · 刚体逆变换 (rigid-body inverse transform)

这一步只换坐标系,不动世界点。想象一下:车在 Δt 内向前挪了 Δy_b 、向侧滑了 Δx_b 、转了 $\Delta \psi$;可是赛道还在那儿,中心线一点没动。于是同一个世界点在新的 body 系下的坐标只是个老坐标系到新坐标系的变换:

$$w'_i = R(-\Delta \psi) (w_i - (\Delta x_b, \Delta y_b))$$

代码里把这一步拆开来写了(去归一化 → 平移 → 旋转,见第 76-86 行),效果等价于上式。

问题:槽位语义已经漂移了。Step 1 之后,第 i 个槽位的 y_b 坐标大约等于 $s_i + \Delta y_b$,也就是说原本代表“前方 20 米”的那个点,现在看起来在前方 18.5 米的位置。如果就地停下,这就是 v5 的做法——100 步以后这些点全跑到车后面去了。

5. Step 2 · 重采样到固定 s (RESAMPLE, 核心贡献)

目标:让槽位 i 的 y_b 重新等于 s_i 。我们手上有 10 个 w'_i 已经在新 body 系里,它们的 y_b 大致在 $s_i + \Delta y_b$ 这条阶梯上,均匀间隔 Δs 。对每个槽位 i ,目标点 ($y_b = s_i$) 恰好落在 w'_i 与 w'_{i+1} 之间。用线性插值把它捞出来:

$$\widetilde{w}_i = (1 - \alpha) w'_i + \alpha w'_{i+1}, \quad \alpha = \frac{\Delta y_b}{\Delta s}$$

为什么 $\alpha = \Delta y_b / \Delta s$?

Step 1 之后,槽位 i 的 y_b 是 $s_i + \Delta y_b$,槽位 $i+1$ 的 y_b 是 $s_{i+1} + \Delta y_b = s_i + \Delta s + \Delta y_b$ 。两点之间走的纵向距离是 Δs (还是这个间距,刚体变换不改变距离)。我们要从 $(s_i$

Δy_b) 走到目标 s_i , 需要前进 Δy_b 这么远; α 就是“前进 $\Delta y_b \div$ 总距 Δs ”, 所以 $\alpha = \Delta y_b / \Delta s$ 。因为 $\Delta y_b \ll \Delta s$ (50 FPS, 典型 $\Delta y_b \approx 0.5 - 1.5$, 而 $\Delta s = 5$), α 始终很小, 插值落在两端之间, 数值很稳。

6. Step 3 · 前端槽位的切线外推

上面的插值需要槽位 $i+1$ 。最远那个槽位 $i = N-1 = 9$ 没有 $i+1$, 因为它已经是“前方 40”这个最远的采样点了。于是用最后两个 waypoint (w'_8, w'_9) 估出车道在该处的切线方向, 然后沿切线再走 α 这么多:

$$\widetilde{w}_9 = w'_9 + \alpha(w'_9 - w'_8)$$

几何上, 这等价于假设车道在末端一小段近似为直线, 先沿着这条直线再前进 $\alpha \cdot \Delta s$ 。由于 α 通常 $\ll 1$, 外推距离很短(几厘米到几十厘米), 误差完全可以忽略。

7. Step 4 · 0.1 倍 MLP 学习残差

前三步是纯几何, 基于两条假设: (1) 车道局部接近直线; (2) 车一步 $\Delta y_b \ll \Delta s$ 。对真实赛道, 这两条只是近似: 中心线有曲率; 过弯时车身漂移、视觉透视等都会让“纯几何”预测略微偏离真值。于是再加一个小的可学习修正:

$$w_i^{t+1} = \widetilde{w}_i + 0.1 \cdot \text{MLP}_{\text{res}}(z_{t+1}^{\text{car}}, \widetilde{w}_{0..9}^{\text{norm}}, a_t)$$

0.1 这个系数很关键。它强制 MLP 只能做小范围微调, 不可能推翻几何步的结果。整套 dynamics 是“几何主导, 网络补差”——这正是 PIWM “Physics-Informed” 名字的由来。

8. 完整数值示例

取一个具体一步 ($\Delta t = 0.02\text{ s}$): $\Delta s = 5, \Delta y_b = 1.5, \Delta x_b = \Delta \psi = 0$ (为了把视线集中到 y_b 上,省去横向位移与转动)。于是 $\alpha = 1.5 / 5 = 0.3$ 。

slot i	s_i (= 目标 y_b)	Step 1 后 y_b	Step 2 后 y_b
0	-5	-6.5	-5.0
1	+0	-1.5	+0.0
2	+5	+3.5	+5.0
3	+10	+8.5	+10.0
4	+15	+13.5	+15.0
5	+20	+18.5	+20.0
6	+25	+23.5	+25.0
7	+30	+28.5	+30.0
8	+35	+33.5	+35.0
9	+40	+38.5	+40.0

可以看到,经过 Step 2 之后,“Step 2 后 y_b” 一栏与“目标 y_b” 一栏完全一致。这就是 RESAMPLE 的魔法:无论你是 1 步还是 100 步之后,槽位 i 永远代表“沿路前方 s_i 处”,语义不漂移。

9. 可视化

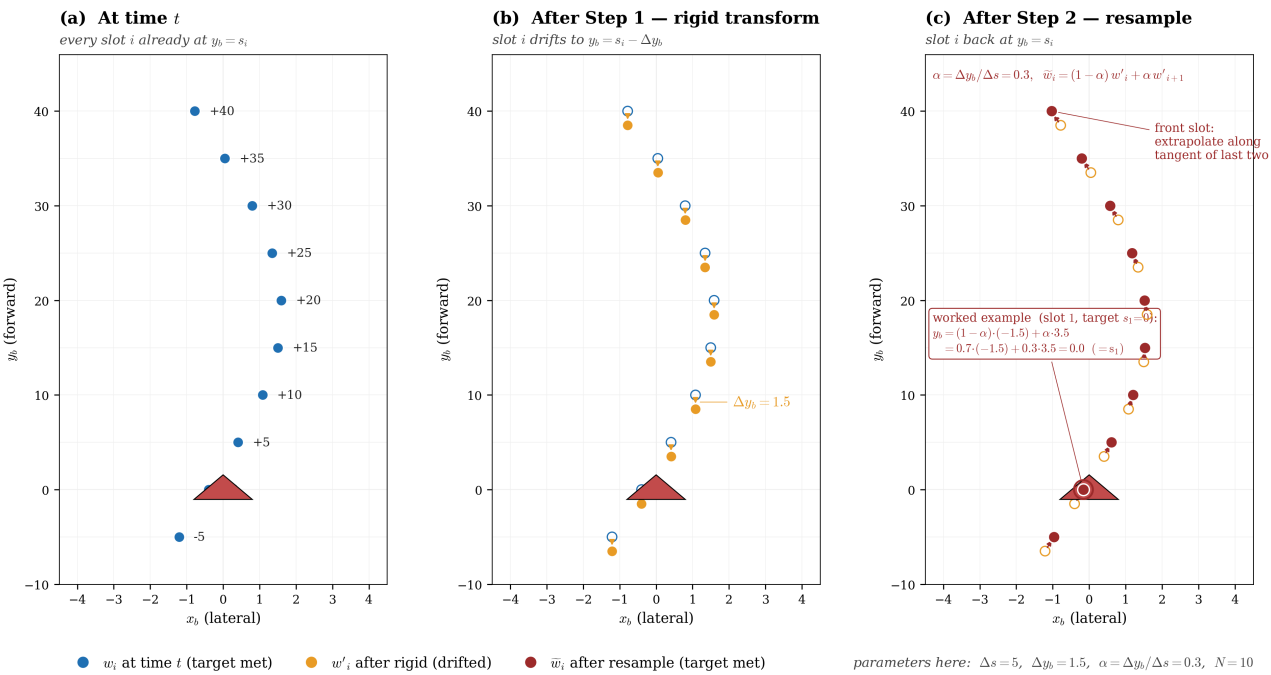


图 1 • 重采样三步走。(a) t 时刻, 每个槽位都已经在 $y_b = s_i$; (b) Step 1 后, 所有点 y_b 整体下移 Δy_b , 槽位语义漂移; (c) Step 2 后, 经线性插值, 槽位 i 重新回到 $y_b = s_i$ 。图中以具体数字 $\Delta y_b = 1.5$, $\alpha = 0.3$ 为例。

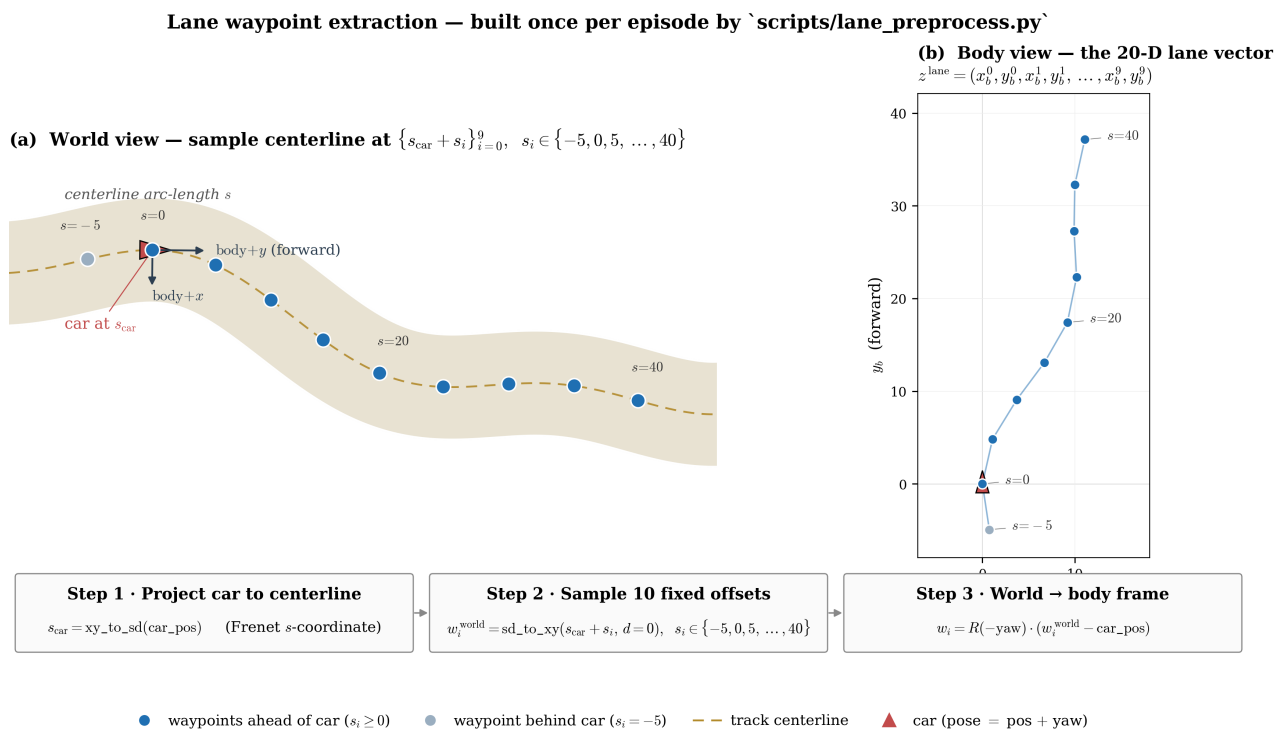


图 2 • waypoint 是怎么取的。每帧把车投影到中心线得到 s_{car} , 在 $s_{car} + s_i$ 处采样中心线 ($d=0$), 再旋转-平移到当前 body 系。

10. 假设、限制与公平性

(a) 几何步的假设。Step 2 / Step 3 都是“相邻 waypoint 之间是直线”的近似。采样间距 $\Delta s = 5$ 远小于赛道曲率半径, 这个近似在 CarRacing 上误差很小; Step 4 的 MLP 残差专门负责修这个近似误差。

(b) Δy_b 必须远小于 Δs 。 $\alpha = \Delta y_b / \Delta s$ 应当落在 $(0, 1)$ 之间。50 FPS 下 $\Delta t = 0.02$ s, 即使车以 100 单位/秒(已经飙得很快)前进, $\Delta y_b \approx 2.0$, 仍然有 $\alpha \approx 0.4$, 远没到失稳。如果未来要把帧率降到 5 FPS, 就需要在两个相邻采样点之间插更多 waypoint, 或者一次跨多个槽位。

(c) 训练时 vs 推理时。训练阶段会用 GT 轨道(road_poly)+ GT 车位姿离线计算 lane_wp_body, 作为 encoder 的监督标签(类似分类问题中的 label)。推理时, 网络只看图像和动作, 不接触任何 GT 轨道: encoder 从 15 帧叠加里读出 20 维 lane, 然后 dynamics 块按上述四步滚动。这与“GT 轨道作为输入特征”是两件完全不同的事。

(d) baseline 的公平性。为了和 PIWM 比, DVBF / GOKU / V2P / SINDy-C 都被改造成同样吃 29 维 latent, 用同样的 encoder、同样的 lane 监督; 只在 dynamics 替换。见 baselines/shared_dynamics_lane.py 与 baselines/sindyc_lane.py。v6 的优势仅来自“如何把这 20 维往前滚”这个 dynamics 设计本身。

11. 一句话总结

每一步 lane 预测 = 换坐标系(Step 1) + 沿折线前进 $\alpha = \Delta y_b / \Delta s$ 的步长重采样(Step 2) + 末端切线外推(Step 3) + $0.1 \times$ 神经网络微调(Step 4)。前三步保证“槽位 i 永远代表沿路 s_i 处”这个语义不漂移, 第四步把几何近似的误差补回来。